Machine Learning & Deep Learning: Comprehensive Course Notes

Unit 1: Foundations of Machine Learning — Supervised, Unsupervised, and Statistical Learning

1.1 Definition & Paradigms of Learning

Machine Learning (ML) is the computational study of algorithms that improve their performance at specific tasks through empirical data rather than rule-based programming (Tom Mitchell, 1997). The three primary learning paradigms are:

- **Supervised Learning:** The algorithm learns a mapping function f: X → Y from labeled input-output pairs. Primary tasks include Regression (predicting continuous outputs like house prices or temperature) and Classification (predicting categorical labels like spam detection or disease diagnosis).
- **Unsupervised Learning:** The model uncovers latent geometric structures, probability distributions, or clusters in unlabeled data X without supervisor targets. Key techniques include Clustering (K-Means, DBSCAN, Hierarchical) and Dimensionality Reduction (Principal Component Analysis / PCA, t-SNE, UMAP).
- **Reinforcement Learning (RL):** An autonomous agent learns optimal policies π(a|s) through sequential trial-and-error interactions with a dynamic environment to maximize cumulative discounted reward signals.

1.2 The Generalization Problem & Bias-Variance Tradeoff

The central goal of machine learning is generalization: achieving low risk on unseen data drawn from the same underlying distribution. The total expected generalization error decomposes into three additive terms:

Total Expected Error = Bias² + Variance + Irreducible Error (σ²)

- **High Bias (Underfitting):** The learning algorithm makes overly simplistic assumptions about the target function, failing to capture true non-linear patterns (e.g., fitting a straight line to sinusoidal data). Both training and test errors are high.
- **High Variance (Overfitting):** The model is excessively flexible and fits statistical noise or sample-specific idiosyncrasies rather than the true underlying distribution. Training error is very low, but validation/test error is high.
- **Regularization:** Penalizing model complexity to control variance. L1 Regularization (Lasso, adds λ||w||■) drives uninformative weights strictly to zero, performing automated feature selection. L2 Regularization (Ridge / Weight Decay, adds λ||w||■²) shrinks weight magnitudes toward zero, stabilizing multicollinearity.

1.3 Core Classical Machine Learning Algorithms

- **Linear & Logistic Regression:** Linear regression models E[Y|X] = Xw via Ordinary Least Squares (OLS) or Gradient Descent. Logistic regression models the probability of binary classes using the Sigmoid link function: P(Y=1|X) = 1 / (1 + e^(-Xw)), trained via Binary Cross-Entropy (Log-Loss).
- **Decision Trees & Ensembles:** Decision Trees recursively partition feature space by maximizing Information Gain (entropy reduction) or Gini Impurity. **Random Forests** aggregate decorrelated decision trees via Bootstrap Aggregation (bagging), drastically reducing variance. **Gradient Boosting Machines (XGBoost, LightGBM)** iteratively train weak learners to minimize pseudo-residuals of the existing ensemble.
- **Support Vector Machines (SVM):** Finds the optimal separating hyperplane that maximizes the geometric margin between decision boundaries and support vectors. Non-linear classification is enabled by the Kernel Trick (e.g., Radial Basis Function / RBF kernel), mapping inputs into infinite-dimensional Hilbert spaces without explicit coordinate transformations.

Unit 2: Neural Networks & Deep Learning Fundamentals

Artificial Neurons, Non-Linear Activations, Backpropagation, and Optimization

2.1 The Artificial Neuron & Multilayer Perceptrons

Artificial Neural Networks (ANNs) consist of interconnected processing units modeled after biological neurons. A single neuron calculates a weighted linear combination of inputs followed by a scalar non-linear activation function:

$z = \sum (w_i \cdot x_i) + b = w^T x + b$, $a = \sigma(z)$

The **Universal Approximation Theorem** (Cybenko, 1989; Hornik, 1991) establishes that a feedforward network with a single hidden layer containing finite non-linear neurons can approximate any continuous function on compact subsets of R^n to arbitrary precision.

2.2 Non-Linear Activation Functions

Activation functions inject non-linearity, allowing deep networks to compute non-linear manifolds and composite representations:

- **Sigmoid (Logistic):** $\sigma(z) = 1 / (1 + e^{-z})$. Maps inputs to range (0, 1). Suffers severely from the vanishing gradient problem because derivatives approach zero for $|z| > 4$.
- **Hyperbolic Tangent (Tanh):** $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$. Zero-centered output in range (-1, 1). Better than sigmoid for intermediate layers, but still saturates at extremes.
- **Rectified Linear Unit (ReLU):** $f(z) = \max(0, z)$. The industry standard for hidden layers. It eliminates vanishing gradients for positive inputs, enables sparse activations, and computes exceptionally fast. The 'Dying ReLU' problem occurs when neurons output 0 for all inputs due to large negative biases.
- **Leaky ReLU & GELU:** Leaky ReLU introduces a small positive slope (e.g., 0.01z) for $z < 0$. Gaussian Error Linear Unit (GELU) weights inputs by their normal cumulative distribution, utilized in modern transformer models like BERT and GPT.
- **Softmax:** Used in categorical multi-class output layers: $P(Y=k|z) = e^{z_k} / \sum e^{z_j}$, transforming raw logit scores into a normalized probability distribution summing to 1.0.

2.3 Loss Functions & Backpropagation

Neural network parameters (weights W and biases b) are optimized by minimizing an empirical risk loss function $L(y_{true}, y_{pred})$. Common loss functions include **Mean Squared Error (MSE)** for continuous targets and **Categorical Cross-Entropy Loss**: $L = -\sum y_k \log(y_{pred_k})$ for classification.

Backpropagation is an exact and efficient implementation of the multivariate calculus chain rule. For each layer l, the error gradient $\delta^l = \partial L / \partial z^l$ is recursively propagated backwards from output layer L to input layer 1:

$\delta^l = ((W^{(l+1)})^T \cdot \delta^{(l+1)}) \blacksquare \sigma'(z^l)$, $\partial L / \partial W^l = \delta^l \cdot (a^{(l-1)})^T$

2.4 Gradient-Based Optimization Algorithms

- **Stochastic Gradient Descent (SGD):** Updates parameters using minibatches of data: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta L_B$. Fast, but oscillates in ravines of ill-conditioned curvature.
- **Momentum:** Accumulates an exponentially decaying moving average of past gradients ($v_t = \gamma v_{(t-1)} + \eta \nabla L$), dampening high-frequency oscillations and accelerating through saddle points.
- **Adam (Adaptive Moment Estimation):** Combines Momentum (first moment estimation m_t) and RMSprop (second moment uncentered variance estimation v_t) with bias corrections: $\theta \leftarrow \theta - (\eta / (\sqrt{v\blacksquare_t} + \varepsilon)) \cdot m\blacksquare_t$. Adam is the default optimizer for deep learning.

Unit 3: Modern Deep Learning Architectures

Convolutional Neural Networks, Recurrent Networks, and the Transformer Revolution

3.1 Convolutional Neural Networks (CNNs) for Spatial Data

CNNs exploit spatial translation invariance and local pixel correlations through parameter sharing and sparse connectivity, drastically reducing parameter counts compared to dense networks:

- **Discrete Convolution Operation:** A small learnable tensor kernel K of size (k_h × k_w) slides across the input tensor X with stride s and zero-padding p, computing localized dot products: S(i,j) = (X * K)(i,j) = ∑ ∑ X(i-m, j-n) K(m,n).
- **Receptive Field:** The specific region of the input image that directly influences the activation of a deeper neuron. Deeper layers aggregate larger spatial contexts.
- **Pooling Layers:** Max-Pooling and Average-Pooling downsample spatial feature maps, providing minor spatial translation invariance and reducing memory consumption.
- **Residual Networks (ResNets):** Introduced **Skip Connections (Identity Shortcuts)**: y = F(x, {W_i}) + x. By allowing gradients to flow unimpeded directly through the identity channel, ResNets resolved vanishing gradients and enabled training networks of 100+ layers.

3.2 Recurrent Neural Networks (RNNs) & LSTMs for Temporal Sequences

Standard RNNs model sequential data (text, time series) by maintaining an internal hidden state h_t updated at each discrete time step: h_t = tanh(W_hh · h_(t-1) + W_xh · x_t + b). However, backpropagation through time (BPTT) causes exponential gradient explosion or vanishing over long sequences.

Long Short-Term Memory (LSTM) Networks solve long-term temporal dependencies through an explicit Cell State C_t and three multiplicative gating mechanisms:

- **Forget Gate:** f_t = σ(W_f · [h_(t-1), x_t] + b_f) determines what proportion of prior cell memory to discard.
- **Input Gate & Candidate State:** i_t = σ(W_i · [h_(t-1), x_t] + b_i) and C■_t = tanh(W_c · [h_(t-1), x_t] + b_c) decide which new candidate information to write to memory.
- **Cell State Update:** C_t = f_t ■ C_(t-1) + i_t ■ C■_t (linear additive gradient highway).
- **Output Gate:** o_t = σ(W_o · [h_(t-1), x_t] + b_o); h_t = o_t ■ tanh(C_t) determines the new emitted hidden state.

3.3 The Transformer & Self-Attention Mechanism

Introduced in 'Attention Is All You Need' (Vaswani et al., 2017), the Transformer discarded sequential recurrence entirely in favor of parallelized Self-Attention mechanisms, powering modern generative AI:

- **Scaled Dot-Product Attention:** Projects each token embedding into Query (Q), Key (K), and Value (V) matrices:

Attention(Q, K, V) = softmax((Q · K^T) / sqrt(d_k)) · V

The scaling factor 1/sqrt(d_k) prevents dot products from growing excessively large in high dimensions, which would cause the softmax function to enter regions with minuscule gradients.

- **Multi-Head Attention:** Computes h distinct attention projections in parallel, allowing the model to jointly attend to information from different representation subspaces (e.g., syntactic vs. semantic relationships).
- **Positional Encoding:** Because self-attention is permutation-invariant, sinusoidal or rotary positional encodings (RoPE) are added to token embeddings to encode sequence order.

3.4 Autoencoders, Encoders & Decoders (Unsupervised Representation Learning)

An Autoencoder is an unsupervised neural network trained to reconstruct its input x at the output layer x■ without labeled supervision, learning efficient compressed representations:

- **Encoder Network z = f_θ(x):** Compresses the high-dimensional input vector x into a low-dimensional latent space representation (bottleneck code z = σ(W_e · x + b_e)). The bottleneck forces the network to strip extraneous noise and preserve only the core, salient features.
- **Decoder Network x■ = g_φ(z):** Reconstructs an approximation of the original input from the compressed latent code (x■ = σ(W_d · z + b_d)).

• **Reconstruction Loss:** The model is trained end-to-end by minimizing reconstruction error: L(x, x■) = ||x - x■||² (Mean Squared Error) for continuous data or Binary Cross-Entropy for normalized inputs.

• **Key Variants & Applications:** Denoising Autoencoders (DAEs) add artificial noise to inputs to learn robust features; Variational Autoencoders (VAEs) map inputs to a probabilistic latent distribution q(z|x) ~ N(μ, σ²) for generative modeling; Undercomplete Autoencoders provide non-linear dimensionality reduction superior to linear PCA.

Unit 4: Model Evaluation, Optimization & Modern Frontiers

Metrics, Normalization, Training Stability, and Foundation Models

4.1 Quantitative Model Evaluation Metrics

Relying solely on Accuracy is misleading for imbalanced datasets. Standard metrics derived from the Confusion Matrix (TP, FP, TN, FN) include:

- **Precision = TP / (TP + FP):** Of all instances predicted positive, how many are truly positive? Critical when false alarms carry high costs (e.g., fraud flags).
- **Recall (Sensitivity) = TP / (TP + FN):** Of all actual positive cases, how many did the model detect? Vital in medical triage and defect detection.
- **F1-Score = 2 · (Precision · Recall) / (Precision + Recall):** The harmonic mean balancing precision and recall.
- **ROC-AUC:** Area Under the Receiver Operating Characteristic Curve plots True Positive Rate vs. False Positive Rate across all classification thresholds, measuring rank-order separation power.

4.2 Deep Network Normalization & Regularization Techniques

- **Batch Normalization (BatchNorm):** Normalizes activations across the mini-batch dimension: x■ = (x - μ_B) / sqrt(σ_B² + ε), scaled and shifted by learnable parameters γ and β. Smooths the optimization landscape and enables higher learning rates.
- **Layer Normalization (LayerNorm):** Normalizes across features for each individual sample independently: x■ = (x - μ_L) / sqrt(σ_L² + ε). Essential for sequential and transformer models where batch sizes vary or are small.
- **Dropout:** Randomly zeroes out a fraction p of hidden units during each forward training pass, preventing complex co-adaptations of feature detectors and acting as an implicit ensemble of 2^N sub-networks.
- **Early Stopping & Learning Rate Schedules:** Monitoring validation loss to halt training before overfitting begins; pairing with Cosine Annealing or Warmup-Decay learning rate schedules for optimal convergence.

4.3 Modern AI Frontiers: Foundation Models & RAG

- **Pre-training & Fine-Tuning Paradigm:** Modern models (LLMs like Llama, Grok, GPT) are pre-trained on massive corpora via self-supervised next-token prediction, followed by Reinforcement Learning from Human Feedback (RLHF) or DPO for alignment.
- **Parameter-Efficient Fine-Tuning (PEFT / LoRA):** Low-Rank Adaptation (LoRA) freezes base model weights W_0 and injects trainable rank-decomposition matrices (W = W_0 + B · A, where r ■ d), cutting trainable parameters by 99% while preserving performance.
- **Retrieval-Augmented Generation (RAG):** Overcomes knowledge cutoff and hallucination by retrieving domain-specific text chunks via high-dimensional vector embeddings and cosine similarity search (pgvector), augmenting the LLM prompt with grounded context.
- **AI Ethics & Responsible AI:** Detecting demographic bias in training corpora, mitigating model hallucination, ensuring explainability, and defending against adversarial prompt injections.